

## General Disclosure

I used “GPT 5.6 Sol Think” within Copilot.

I consulted it on 9/26/2026.

## Code Attribution

This is my first prompt. I included instructions and testcases from the assignment, and I also told the AI one way it could approach the problem and that it should report back to me with its ideas.

*Problem description* Write a C or C++ program that reads a line of text and extracts a single valid IPv4 address — optionally followed by a port number — embedded anywhere in that text. Only digits, periods (.), and colons (:) are ever part of a valid token; every other character is garbage and is skipped. A candidate token must match the address grammar in full — no partial matches, no truncating to find a valid piece inside a longer run. An address is four octets separated by periods (octet.octet.octet.octet), each octet 1–3 digits, value 0–255, no leading zero unless the value is exactly 0. An optional :port may follow the fourth octet: 1–5 digits, value 0–65535, same leading-zero rule. If a colon is present, the port must be fully valid or the entire match — address included — is rejected.

*Function prototype* The function returns whether a valid address was found, and delivers the address and port through two additional parameters. C: // Returns 1 if a valid address was found, 0 otherwise. // On success: \*outAddress holds the 32-bit value, and // \*outPort holds the port number, or -1 if no port was present. // On failure: \*outAddress is set to 0 and \*outPort is set to -1. int extractIPv4(const char\* str, unsigned long\* outAddress, int\* outPort); C++:// Returns true if a valid address was found, false otherwise. // On success: outAddress holds the 32-bit value, // and outPort holds the port number, or -1 if no port was present. // On failure: outAddress is set to 0 and outPort is set to -1. bool extractIPv4(const std::string& str, unsigned long& outAddress, int& outPort);

*Built-in functions/libraries you may not use* Any string-to-number conversion function: atoi, atol, atoll, strtol, strtoul, strtod, stoi, stol, stoul, sscanf, scanf with numeric conversions. Any address-parsing library function: inetaton, inetpton, inet\_addr, or equivalents. Any regular-expression facility (std::regex, POSIX regex.h, or similar) — the parsing and validation logic must be your own character-by-character code, not a pattern matched by a library. Standard character-classification functions (isdigit, etc.) are fine to use.

*Other requirements* Exactly one valid address may be extracted per input line; everything else in the line is either garbage (skipped) or part of a candidate token that fails validation. Reject anything that does not exactly match the

grammar above — wrong octet count, empty octet, out-of-range octet or port, a disallowed leading zero, a second colon, a colon not immediately after the fourth octet, or a stray period/colon directly adjacent to an otherwise-valid address. On success, print: Extracted IPv4 address: A.B.C.D (decimal value: N, port: P) where N is the 32-bit decimal value and P is the port number or the literal text none. Program requirements Input loop: main continuously prompts the user for input until the user enters END (case-sensitive), then prints Program terminated. and exits. Extraction function: implement extractIPv4 exactly as prototyped above. All digit accumulation must be done by hand. Display: main receives the result from extractIPv4 and formats the output exactly as specified above. Sample run Enter a string (or 'END' to quit): connecting to 192.168.1.1 now Extracted IPv4 address: 192.168.1.1 (decimal value: 3232235777, port: none) Enter a string (or 'END' to quit): server=10.0.0.255:8080end Extracted IPv4 address: 10.0.0.255 (decimal value: 167772415, port: 8080) Enter a string (or 'END' to quit): 192a168.1.1.1 Extracted IPv4 address: 168.1.1.1 (decimal value: 2818638081, port: none) Enter a string (or 'END' to quit): 192.168.1.1. Invalid input: no valid IPv4 address found Enter a string (or 'END' to quit): Connection from 192.168.1.1 refused Extracted IPv4 address: 192.168.1.1 (decimal value: 3232235777, port: none) Enter a string (or 'END' to quit): 192.168.01.1 Invalid input: no valid IPv4 address found Enter a string (or 'END' to quit): 1.2.3.4:99999 Invalid input: no valid IPv4 address found Enter a string (or 'END' to quit): 12.34.56 Invalid input: no valid IPv4 address found Enter a string (or 'END' to quit): no number here Invalid input: no valid IPv4 address found Enter a string (or 'END' to quit): END Program terminated. Plan how you will write a C++ program for this. My idea is that it should get all contiguous strings of 0...9, ., and :, and then perform the validation on each of these and return any valid one. Determine whether this is good for the first approach, as well as how the validation should be done.

This is my second prompt. I approved its ideas and then told it to come up with edge cases and then write the code.

*Come up with edge cases to look out for, and then write the program.*

All the code is AI-generated (but the comments within `/*...*/` are written by me), as the AI's first iteration was able to pass all testcases I gave it. I gave it some test cases I came up with that had mixtures of almost valid and valid IP addresses and ports (like "127.0.0.1:90000helloworld127.0.0.2:90"), as well as test cases from the assignment description's sample runs.

The only modification I made to it was adding the comments within `/*...*/` to include attributions within the code.

**Verification Statement**

I understand every line of the submitted code.

The code has been tested and works as intended.

I do not know of any unresolved bugs, limitations, or unexpected behaviors.